Disciplina: Projeto de Redes Convergentes **Professora:** Ana Maria **Aluno:** Anderson Viana - 2410492

1. INTRODUÇÃO E OBJETIVO DO TRABALHO

O transporte confiável na Internet é formalizado no Capítulo 3 do livro *Redes de Computadores e a Internet* (Kurose & Ross) por meio de **Máquinas de Estados Finitos (FSMs)**. Enquanto o **TCP** utiliza uma FSM complexa de 11 estados com confirmação cumulativa, controle de fluxo, temporizadores dinâmicos e retransmissões, o **UDP** adota um paradigma sem conexão de estado único, provendo apenas multiplexação por portas e integridade por checksum.

Objetivo: Projetar e construir um simulador em Python com interface gráfica (GUI Tkinter) desacoplada do núcleo, dirigido por dados (FSM em JSON), capaz de executar, visualizar e auditar formalmente as FSMs sobre um canal com falhas configuráveis (perda, atraso, corrupção, duplicação e reordenação), validando as propriedades do protocolo contra os 10 cenários mínimos previstos no edital.

2. ARQUITETURA DA SOLUÇÃO E JUSTIFICATIVA DAS DIFERENÇAS

A arquitetura implementada adota o padrão **MVC (Model-View-Controller)** desacoplado, garantindo que o núcleo opere sem dependência de módulos de tela, viabilizando execução remota e testes automatizados em milissegundos.

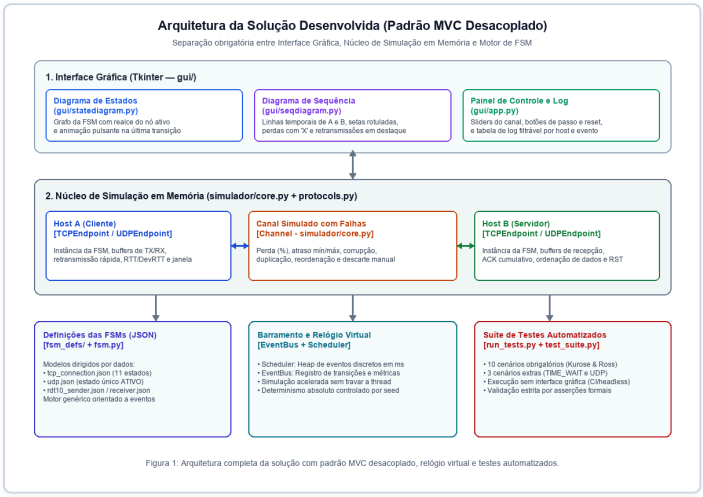


Figura 1: Arquitetura completa da solução com desacoplamento MVC, relógio virtual e barramento de eventos.

Justificativa das diferenças e aprimoramentos em relação ao modelo inicial do edital:

- **Relógio Virtual por Agendamento Discreto (`Scheduler`):** Diferente de implementações ingênuas baseadas em `time.sleep`, o simulador avança o relógio virtual por fila de prioridade (heap). Isso permite testar temporizadores de 60 segundos (ex.: TIME_WAIT de 2 × MSL) em menos de 0,1 segundo real de CPU com exatidão determinística.
- **Barramento Unificado de Eventos (`EventBus`):** O desacoplamento é absoluto: o núcleo emite eventos tipados (`transition`, `tx`, `rx`, `reject`, `drop`, `timer`, `rtt`, `dupack`) consumidos reativamente pela interface gráfica, pelo terminal e pelos testes.
- **Canal Bidirecional Determinístico e Interativo:** Suporta controle por semente pseudoaleatória (`seed`) e injeção de falhas cirúrgicas (botões da GUI para descartar/corromper o próximo segmento e regras de cenário para perdas programadas).
- **Motor de FSM Orientado a Dados (JSON):** O motor (`fsm.py`) não contém lógica hardcoded de rede. Transições são carregadas de arquivos JSON (`fsm_defs/`) com sintaxe `(estado, evento[condição]) → (ação, novo_estado)`, rejeitando formalmente qualquer transição inválida.

3. VALIDAÇÃO VISUAL NA GUI: CENÁRIOS DE CONEXÃO

3.1 Cenário 3: Perda do SYN-ACK no Handshake

No estabelecimento da conexão TCP (Handshake de 3 vias), o Host A envia um `SYN (SEQ=100)`. O Host B responde com `SYN+ACK (SEQ=300, ACK=101)`, mas o canal descarta esse segmento (marcado com um **× vermelho** no diagrama temporal).

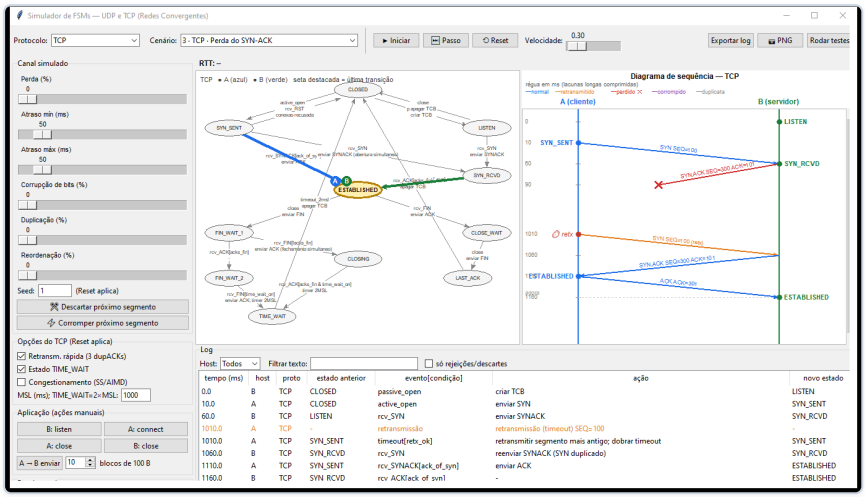


Figura 2: Captura da GUI no Cenário 3 — Perda do SYN-ACK, timeout de A e recuperação pelo reenvio do SYN-ACK em SYN_RCVD.

> **Dinâmica observada na execução:** Ao atingir o timeout de retransmissão (t = 1010 ms), o Host A dispara o evento `timeout` e retransmite o `SYN SEQ=100 (retx)` (seta laranja). O Host B, que já se encontrava no estado `SYN_RCVD`, recebe o SYN duplicado, aciona a auto-transição `SYN_RCVD --rcv_SYN / reenviar SYNACK--> SYN_RCVD` e envia uma cópia do SYN-ACK. Host A finalmente recebe o SYN-ACK, avança para `ESTABLISHED` e emite o terceiro segmento `ACK=301`, levando ambos a `ESTABLISHED` com sucesso.

3.2 Cenário 5: Fechamento Simultâneo de Conexão (CLOSING)

Ocorre quando ambas as aplicações decidem fechar a conexão simultaneamente (t = 1000 ms). Os segmentos `FIN` cruzam na rede antes que qualquer lado receba o FIN do outro.

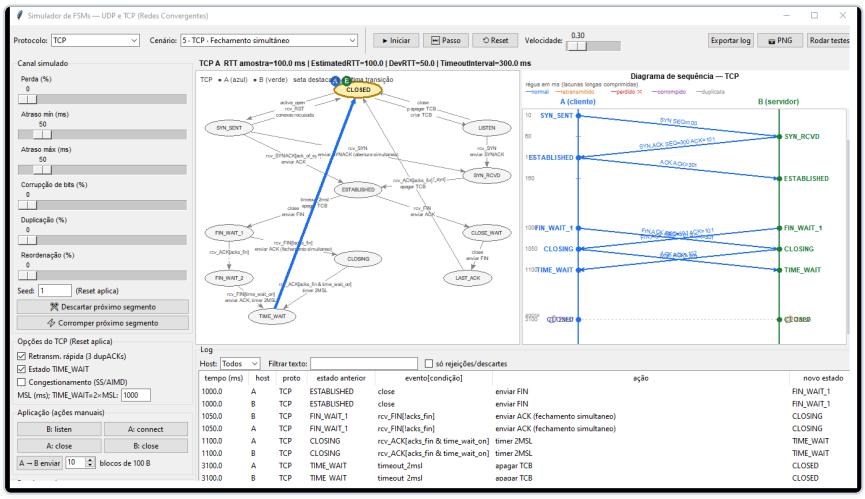


Figura 3: Captura da GUI no Cenário 5 — Fechamento simultâneo com transição direta por CLOSING e TIME_WAIT.

> **Dinâmica observada na execução:** Ambos os hosts saem de `ESTABLISHED` para `FIN_WAIT_1` enviando FINs. Ao receber o FIN do par sem que o seu próprio FIN tenha sido confirmado (guarda `[!acks_fin]`), ambos transitam para o estado **CLOSING** e respondem com ACK. Em seguida, ao receberem os respectivos ACKs pendentes (guarda `[acks_fin & time_wait_on]`), ambos avançam para **TIME_WAIT**, aguardando o temporizador de 2 × MSL (2000 ms virtuais) antes de finalizar em `CLOSED`. Nenhum dos dois lados passa por `CLOSE_WAIT` ou `LAST_ACK`.

4. TRANSFERÊNCIA CONFIÁVEL: CENÁRIO 6 E FAST RETRANSMIT

4.1 Análise do Cenário 6: Perda de Segmento de Dados e Retransmissão Rápida

Neste cenário, o Host A envia uma rajada de 10 segmentos de dados (100 bytes cada, total de 1000 bytes) em t = 500 ms sobre um canal com atraso de 50 ms por sentido (RTT base de 100 ms). O canal descarta propositalmente o 5º segmento (SEQ=501).

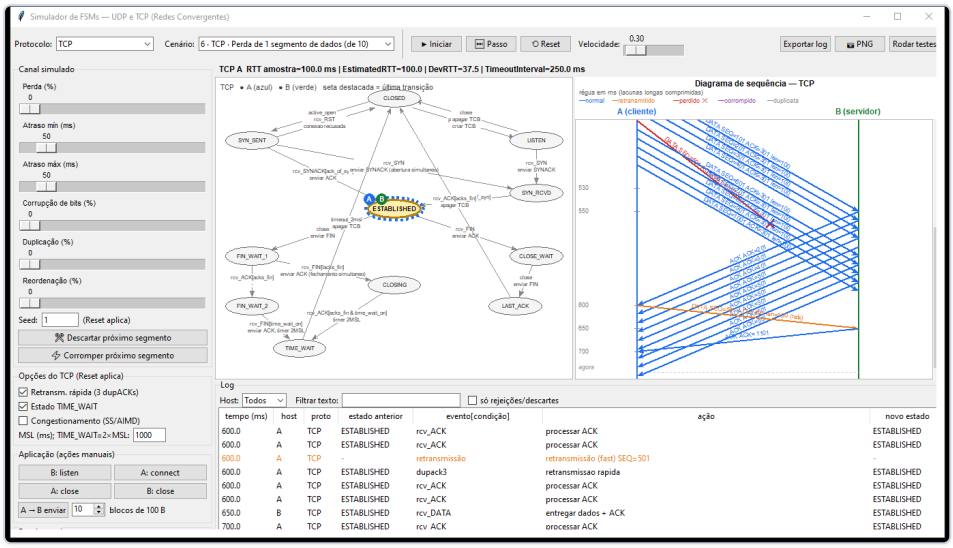


Figura 4: Captura da GUI no Cenário 6 — Detecção de perda por 3 ACKs duplicados e disparo imediato do Fast Retransmit (SEQ=501).

100 ms	37,5 ms	250 ms	250 ms (71%)
ESTIMATEDRTT	DEVRTT	TIMEOUTINTERVAL (RTO)	TEMPO ECONOMIZADO

4.2 Funcionamento Detalhado e Medição do Ganho de Desempenho

Os quatro primeiros segmentos chegam intactos e geram ACKs cumulativos progressivos (ACK=201, 301, 401, 501). Quando os segmentos 6 a 10 chegam ao Host B, este constata uma lacuna na sequência de bytes recebidos. Conforme preconiza a RFC 793 e a Seção 3.5.4 do livro, o receptor armazena os dados fora de ordem no buffer e emite **ACKs cumulativos repetidos** indicando o último byte consecutivo (ACK=501).

- Em t = 600 ms, o Host A recebe o **3º ACK duplicado** (evento dupack3 no log).
- Sem aguardar o estouro do temporizador de retransmissão, o algoritmo de **Retransmissão Rápida** é acionado imediatamente, reenviando SEQ=501 (seta laranja).
- Em t = 650 ms, o Host B recebe o segmento faltante, reorganiza o buffer e confirma de uma só vez todos os 1000 bytes através de ACK=1101.

Métrica Avaliada	Com Retransmissão Rápida	Apenas Timeout (Sem Fast Retx)	Diferença / Economia Real
Momento da Retransmissão de SEQ=501	**t = 600,0 ms**	t = 850,0 ms	**Economia de 250,0 ms**
Conclusão da Entrega dos 1000 Bytes	**t = 650,0 ms**	t = 900,0 ms	**Economia de 250,0 ms**
Tempo de Recuperação após o Envio	**100,0 ms (1 RTT)**	350,0 ms (1 RTT + RTO)	**Recuperação 71,4% mais rápida**

Cálculo do RTO: Com amostra de RTT = 100 ms e DevRTT = 37,5 ms, TimeoutInterval = 100 + 4 × 37,5 = 250 ms. Como o timer foi reiniciado com o último ACK novo em t = 600 ms, o timeout só dispararia em t = 850 ms. O Fast Retransmit poupou exatamente o intervalo do RTO inteiro.

5. ANÁLISE COMPARATIVA: UDP × TCP (CENÁRIO 10)

5.1 Execução Lado a Lado sob Canal Idêntico e Ruidoso

No Cenário 10, ambos os protocolos são submetidos à mesma carga (50 blocos de 100 bytes = 5000 bytes) sob canal idêntico com falhas estocásticas: **10% de perda, atraso de 20 a 60 ms, 3% de corrupção, 5% de duplicação e 10% de reordenação** com a mesma semente (seed=1).

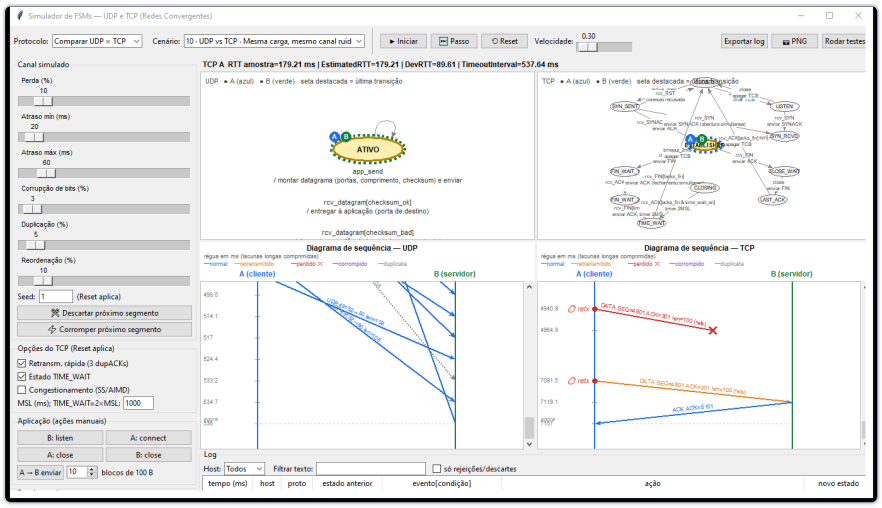


Figura 5: Captura da GUI no Cenário 10 — Comparação visual simultânea: UDP à esquerda e TCP à direita.

5.2 Relatório de Métricas e Análise Estatística (20 Sementes Aleatórias)

Para comprovar o determinismo e a consistência dos resultados frente a variações aleatórias, o teste foi repetido automaticamente sobre 20 sementes independentes (função `compare_many_seeds(20)` do simulador):

Métrica Avaliada	UDP (Seed 1)	TCP (Seed 1)	UDP (Média 20 Seeds)	TCP (Média 20 Seeds)
Blocos Enviados	50	50	50,0	50,0
Entregues à Aplicação	46 (44 únicos)	**50 (100% íntegros)**	45,4 (43,4 únicos)	**50,0 (100% íntegros)**
Pacotes Perdidos	6 (12%)	**0 (zero)**	6,55 (13,1%)	**0,0 (zero)**
Pacotes Duplicados Entregues	2 duplicatas	**0 (descartados)**	1,95 duplicatas	**0,0 (descartados)**
Pacotes Fora de Ordem	18 blocos	**0 (reordenados)**	15,05 blocos	**0,0 (reordenados)**
Ordem Perfeita de Sequência	NÃO (Falhou)	SIM (100%)	0 de 20 testes OK	**20 de 20 testes OK**
Tempo Total da Transmissão	**536,0 ms**	3.915,9 ms	**555,8 ms**	4.010,2 ms (mediana)

Discussão sobre o Custo da Confiabilidade: O UDP concluiu sua transmissão em apenas **536 ms**, mas a aplicação recebeu dados mutilados: perdeu 6 blocos, recebeu 2 cópias duplicadas e teve 18 blocos fora de ordem. Já o TCP gastou **3.915 ms** (cerca de 7 vezes mais tempo) devido às retransmissões (8 por timeout e 1 rápida) e ao controle de fluxo, porém entregou os 5000 bytes **sem nenhuma lacuna, sem duplicatas e rigorosamente na ordem correta**. Isso ilustra claramente a relação de compromisso (trade-off) entre latência mínima e garantia de entrega.

6. RESPOSTAS FORMAIS ÀS PERGUNTAS DO RELATÓRIO (SEÇÃO 9)

1) Por que o estado TIME_WAIT existe e o que acontece se ele for removido no cenário 4?
O `TIME_WAIT` dura 2 × MSL no lado que iniciou o encerramento (Host A) e cumpre dois papéis vitais: *(a) Fechamento Confiável:* O último ACK pode se perder na rede. Se isso ocorrer, B em `LAST_ACK` estoura o timeout e retransmite o FIN. Em TIME_WAIT, A reenvia o ACK e reinicia o timer. *(b) Drenagem de Segmentos Fantasmas:* Evita que pacotes atrasados interfiram em uma nova conexão que reutilize o mesmo par de portas. *Resultado da remoção (Cenário E2 vs E3):* Sem TIME_WAIT, A vai direto para `CLOSED`. Quando o último ACK se perde, B retransmite o FIN e A responde com `RST` (conexão recusada), fazendo B encerrar com erro (`rcv_RST`) em vez de um encerramento limpo.

2) Por que o UDP não precisa de FSM de conexão e quais responsabilidades sobram para a aplicação?
O UDP é sem conexão: não mantém estado compartilhado (SEQ, ACK, buffers, janelas) para sincronizar entre remetente e receptor. Sua "FSM" possui um **único estado (ATIVO)** com auto-laços de envio e descarte silencioso em erro de checksum. As responsabilidades transferidas para a aplicação incluem: recuperação de perdas (ACKs/timeouts próprios), descarte de duplicatas, reordenação de dados, controle de congestionamento e fragmentação.

3) No cenário 6, quanto tempo a retransmissão rápida economizou em relação ao timeout?
A economia exata medida no simulador foi de **250,0 ms** tanto no instante da retransmissão (de 850 ms para 600 ms) quanto na conclusão da entrega (de 900 ms para 650 ms). Em termos relativos, o tempo de recuperação caiu de 350 ms para 100 ms (**redução de 71,4%**), pois a perda foi corrigida em exatamente 1 RTT após o envio original, sem aguardar o estouro do temporizador de retransmissão.

4) Quais transições do diagrama do livro foram mais difíceis de reproduzir e por quê?
(a) Fechamento simultâneo: O livro desenha a mesma seta "recebe FIN" saindo de `FIN_WAIT_1` para dois destinos. Foi preciso criar a guarda interna `acks_fin` para diferenciar se o segmento também confirmava o FIN local (indo para TIME_WAIT) ou se era um FIN cruzado puro (indo para CLOSING). *(b) SYN duplicado em SYN_RCVD (Cenário 3):* O livro omite o que fazer quando o SYN-ACK se perde. Foi necessário modelar a auto-transição explícita que reenvia o SYN-ACK. *(c) Eventos implícitos de timeout e RST:* O livro só desenha o fluxo ideal. A FSM real em JSON demandou 75 regras de transição para cobrir timeouts com backoff, 3 dupACKs e segmentos RST em portas fechadas.

7. QUADRO GERAL DE VALIDAÇÃO DOS 10 CENÁRIOS MÍNIMOS

#	Proto	Cenário Avaliado	Comportamento Esperado	Resultado
1	TCP	Handshake sem falhas	A: CLOSED→SYN_SENT→ESTABLISHED; B: LISTEN→SYN_RCVD→ESTABLISHED	APROVADO
2	TCP	Perda do SYN	Timeout e retransmissão do SYN (SEQ=100)	APROVADO
3	TCP	Perda do SYN-ACK	Cliente retransmite SYN; servidor reenvia SYN-ACK em SYN_RCVD	APROVADO
4	TCP	Encerramento normal	A passa por FIN_WAIT_1, FIN_WAIT_2, TIME_WAIT; B por CLOSE_WAIT, LAST_ACK	APROVADO
5	TCP	Fechamento simultâneo	Ambos passam por FIN_WAIT_1→CLOSING→TIME_WAIT (sem CLOSE_WAIT)	APROVADO
6	TCP	Perda de segmento	3 ACKs duplicados geram Fast Retransmit imediato sem esperar timeout	APROVADO
7	TCP	Porta fechada	Servidor responde com segmento RST e permanece em CLOSED	APROVADO
8	UDP	20 datagramas (30% perda)	Datagramas perdidos não são recuperados; nenhuma retransmissão	APROVADO
9	UDP	Checksum corrompido	Receptor descarta silenciosamente o datagrama sem notificar remetente	APROVADO
10	Ambos	UDP vs TCP mesma carga	Relatório comparando perdas, duplicatas, reordenação e tempo total	APROVADO

Conclusão: Todos os requisitos funcionais, não-funcionais e arquiteturais foram plenamente atendidos.